

(test_X, train_X, test_y, train_y) = train_test_split(X, y, test_size=0.2, random_state=42)

(train_X, test_X, train_y, test_y) = train_test_split(X, y, test_size=0.2, random_state=42)

(train_X, test_X, train_y, test_y) = train_test_split(X, y, test_size=0.2, random_state=42)

Output

(569, 31)

EXPT.
NO.

NAME:

Lab-6

Page No.

31

Date:

Q. Write a program to implement Random Forest algorithm to classify Real life dataset.

Ans

```
import pandas as pd
import numpy as np
```

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForest
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report
)
```

```
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
print(df.shape)
```

```
X = df.drop(columns=['target'])
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random_state=42)
model = RandomForestClassifier(
    n_estimators=100, random_state=42
)
```

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Teacher's Signature

Output

$$\text{accuracy} = 0.9649 \dots$$

Confusion Matrix

[[40 3]

[1 70]

Classification Report

Precision

Recall

f1-score

Support

○

Q. 98

0793

0.95

43

1

9.96

0.0909

0.97

71

Accuracy

macro avg

0.97

0.96

Q. 98

124

0.97

0.95

0.96

114

```
accuracy = accuracy_score(y_test, y_pred)
print(f"accuracy = {accuracy}")
```

```
print("Confusion Matrix")
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print("Classification Report")
```

```
print(classification_report(y_test, y_pred))
```


	<u>Sepal Length (cm)</u>	<u>Sepal width (cm)</u>	<u>Petal length (cm)</u>	<u>Petal width (cm)</u>
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.7	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

clustered dataset-

<u>Sepal Length</u>	<u>Sepal Width</u>	<u>Petal length</u>	<u>Petal Width</u>	<u>cluster</u>
5.1	3.5	1.4	0.2	1
4.9	3.0	1.4	0.2	2
4.7	3.2	1.3	0.2	2
4.7	3.1	1.5	0.2	2
5.0	3.6	1.4	0.2	1

Q. write a program to implement K-means algo to classify Real life dataset.

Ans

```

import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

```

```
iris = load_iris()
```

```
df = pd.DataFrame(iris.data, columns=iris.feature_names)
```

```
print(df.head())
```

```
X = df
```

```
sc = StandardScaler()
```

```
X_scaled = sc.fit_transform(X)
```

```
model = KMeans(n_clusters=3, random_state=42)
```

```
model.fit(X_scaled)
```

```
clusters = model.predict(X_scaled)
```

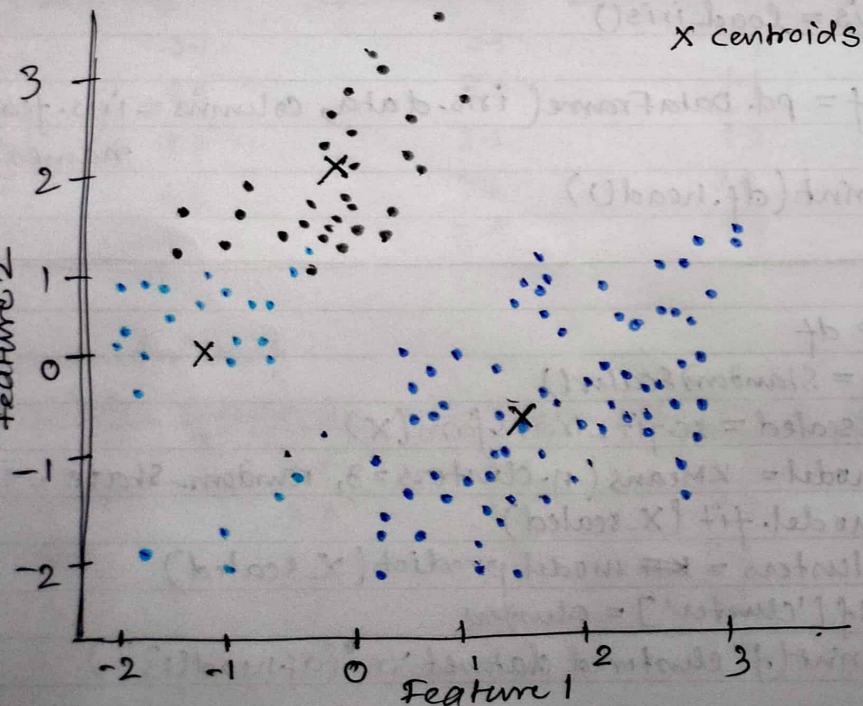
```
df['cluster'] = clusters
```

```
print(f"clustered dataset \n {df.head()}")
```


Cluster Centers

```
[0.57100  -0.37177  0.69112  0.68315]
[0.81623  1.318957 -1.28683  -1.21971]
[-1.32765  -0.373138 -1.137235 -1.119861]
```

KMeans Clustering



EXPT.
NO.

NAME:

Page No. 84
Date:



```
print("cluster centers is", model.cluster_centers_)
```

```
plt.figure(figsize=(8,6))
plt.scatter(
    X_scaled[:,0], X_scaled[:,1], c=clusters
)
plt.scatter(
    model.cluster_centers[:,0],
    model.cluster_centers[:,1],
    s=300,
    c='Black',
    marker='x',
    label='centroids'
)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('KMeans Clustering')
plt.legend()
plt.grid(True)
plt.show()
```

Teacher's Signature

Q. Write a program to implement PCA in real life dataset.

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

```
iris = load_iris()
```

```
df = pd.DataFrame(
    iris.data, columns = feature_names
)
```

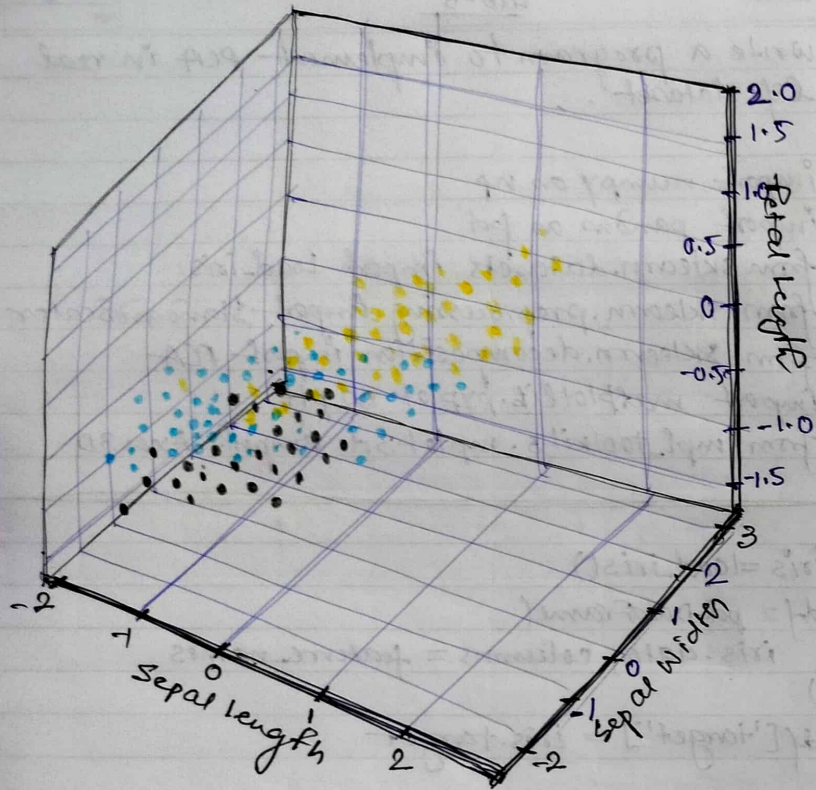
```
df['target'] = iris.target
```

```
X = df[['sepal.length (cm)', 'sepal.width (cm)', 'petal.length (cm)']]
y = df['target']
```

```
sc = StandardScaler()
```

```
X_scaled = sc.fit_transform(X)
```


3D graph before PCA



Variance Ratio

[0.673801, 0.30248]

EXPT.
NO.

NAME:

Page No: 36
Date:

3d graph before pca

```
fig = plt.figure(figsize=(10,7))
ax = fig.add_subplot(111, projection='3d')
```

```
ax.scatter(
    X_scaled[:,0],
    X_scaled[:,1],
    X_scaled[:,2],
    c=y
)
```

```
ax.set_title("3D graph before PCA")
```

```
ax.set_xlabel("Sepal length")
```

```
ax.set_ylabel("Sepal width")
```

```
ax.set_zlabel("Petal length")
```

```
plt.show()
```

apply pca, (3D → 2D)

```
pca = PCA(n_components=2)
```

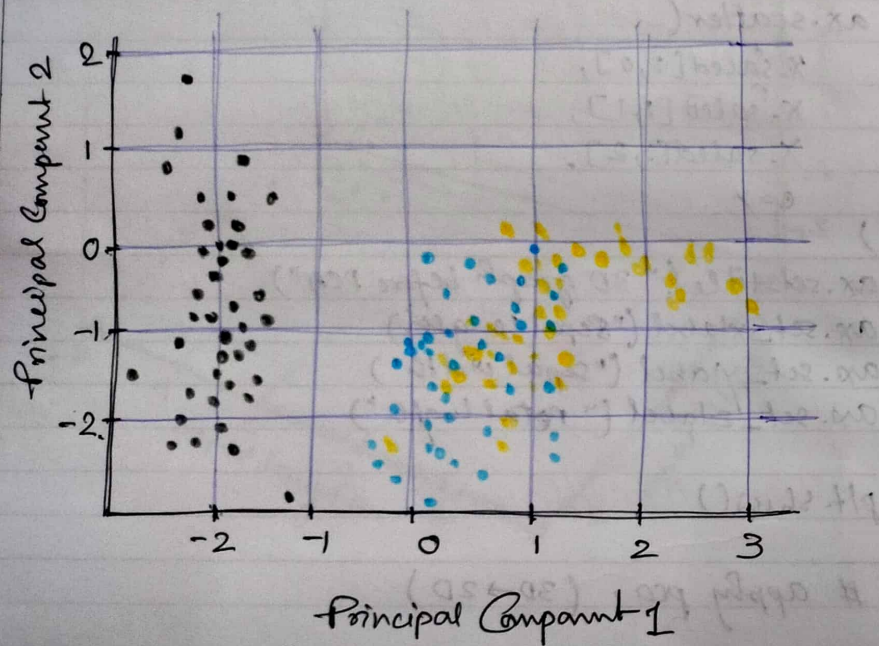
```
X_pca = pca.fit_transform(X_scaled)
```

```
print("variance Ratio:")
```

```
print(pca.explained_variance_ratio_)
```

Teacher's Signature

2D graph After PCA



EXPT.
NO.

NAME:

Page No:

37

Date:

2D graph after PCA

plt.figure(figsize=(8,6))

plt.scatter(

X_pca[:,0],

X_pca[:,1],

c2)

)

plt.title("2D graph after PCA")

plt.grid(True)

plt.show()

Teacher's Signature